

13.1 Recursion

Now that we've done a couple of nested for loops, let's take a look at our favorite problem: recursion.

Consider the recursive function `f3` below:

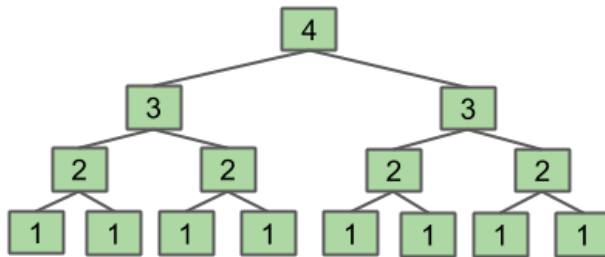
```
public static int f3(int n) {
    if (n <= 1)
        return 1;
    return f3(n-1) + f3(n-1);
}
```

What does this function do?

Let's think of an example of calling `f3(4)`:

- The first call will return $f3(4-1) + f3(4-1)$
- Each $f3(3-1)$ call will branch out to $f3(2-1) + f3(2-1)$
- Then for each $f3(2-1)$ call, the condition `if (n <= 1)` will be true, which will return 1.
- What we observe at the end is that 1 will be returned 8 times, meaning we have $f3(2-1)$ summed 8 times.
- Therefore, $f3(4)$ will return 8.

We can visualize this as a tree, where each level represents a recursive call and each node value represents the argument to the function :

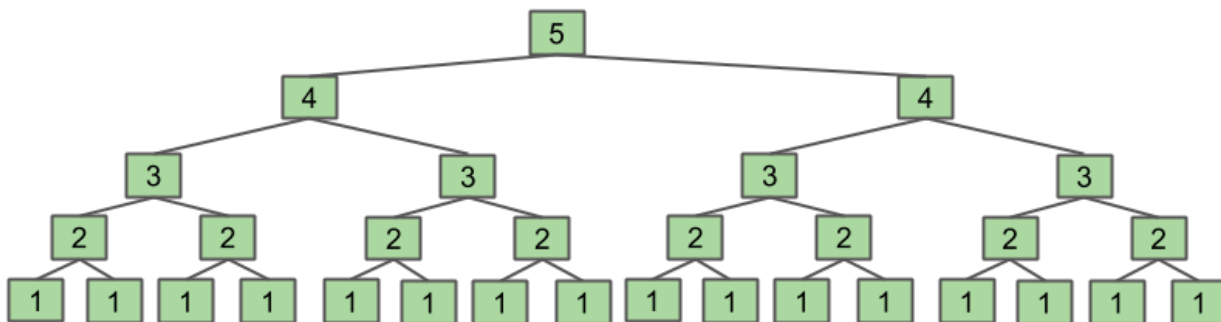


Visualization of `f3`'s recursive calls

You can do a couple more examples, and see that this function returns $2^N - 1$. Visualizing the recursive calls is extremely useful for getting a sense of what the function is doing, and we will discuss a few methods of determining runtime in recursive functions.

Method 1: Intuition

Based on the visualization below, we can notice that every time we add one to n we double the amount of work that has to be done:



Then, adding one to n N times means doubling the amount of work N times, which results in the intuitive answer for runtime to be 2^N .

Method 2: Algebra

Another way to approach this problem is to count the number of calls to `f3` involved. Utilizing the same tree visualization above, we can see that the number of calls to `f3` at each recursive level is equivalent to *the number of nodes at each recursive level* of the tree. For instance, the number of calls we made to `f3` at the top level is 1, at the second level is 2, at the third level is 4, etc.

The total number of calls to `f3` then is the **sum** of the number of nodes at each recursive level, which can be expressed as the equation below:

$$C(N) = 1 + 2 + 4 + \dots + 2^{N-1}$$

Applying the formula we saw earlier for the sum of the first powers of 2:

$$1 + 2 + 4 + 8 + \dots + Q = 2Q - 1$$

Substituting Q with 2^{N-1} , we get:

$$C(N) = 2Q - 1 = 2(2^{N-1}) - 1 = 2^N - 1$$

The work during *each call* is constant, so the overall runtime for `f3` is $\theta(2^N)$.

Method 3: Recurrence Relation (Out of Scope)

This method is not required reading and is outside of the course scope, but worth mentioning for interest's sake.

We can use a “recurrence relation” to count the number of calls, instead of an algebraic approach. This looks like:

$$C(1) = 1 \quad C(N) = 2C(N-1) + 1$$

Expanding this out with a method we will not go over but you can read about in the slides or online, we reach a similar sum to the one above. We can then again reduce it to $2^N - 1$, reaching the same result of $\theta(2^N)$.

13.2 Binary Search

Binary Search Introduction

Binary search is a nice way of searching a list for a particular item. It requires the list to be in sorted order and uses that fact to find an element quickly.

To do a binary search, we start in the middle of the list, and check if that's our desired element. If not, we ask: is this element bigger or smaller than our element?

If it's bigger, then we know we only have to look at the half of the list with smaller elements. If it's too small, then we only look at the half with bigger elements. In this way, we can cut in half the number of options we have left at each step, until we find our target element.

What's the worst possible case? When the element we want isn't in the list at all. Then we will make comparisons until we've eliminated all regions of the list, and there are no more bigger or smaller halves left.

For an animation of binary search, see [these slides](#).

What's the intuitive runtime of binary search? Take a minute and use the tools you know to consider this.

We start with N options, then $N/2$, then $N/4$... until we have just 1. Each time, we cut the array in half, so in the end we must perform a total of $\log_2(N)$ operations. Each of the $\log_2(N)$ operations, eg. finding the middle element and comparing with it, takes constant time. So the overall runtime then is order $\log_2(N)$.

It's important to note, however that each step doesn't cut it *exactly* in half. If the array is of even length, and there is no 'middle', we have to take either a smaller or a larger portion. But this is a good intuitive approach.

We'll do a precise way next.

To precisely calculate the runtime of binary search, we'll count the number of operations, just as we've done previously.

First, we define our cost model: let's use the number of recursive binary search calls. Since the number of operations inside each call is constant, the number of calls will be the only thing varying based on the size of the input, so it's a good cost model.

Like we've seen before, let's do some example counts for specific N . As an exercise, try to fill this table in before continuing:

N	1	2	3	4	5	6	7	8	9	10	11	12	13
Count													

Alright, here's the result:

N	1	2	3	4	5	6	7	8	9	10	11	12	13
Count	1	2	2	3	3	3	3	4	4	4	4	4	4

These seems to support our intuition above of $\log_2(N)$. We can see that the count seems to increase by one only when N hits a power of 2.

...but we can be even more precise: $C(N) = \lfloor \log_2(N) \rfloor + 1$ (These L-shaped bars are the “floor” function, which is the result of the expression rounded down to the nearest integer.)

A couple properties worth knowing (see below for proofs):

- $\lfloor f(N) \rfloor = \Theta(f(N))$
- $\lceil f(N) \rceil = \Theta(f(N))$
- $\log_p(N) = \Theta(\log_q(N))$

The last one essentially states that for logarithmic runtimes, the base of the logarithm doesn’t matter at all, because they are all equivalent in terms of Big-O (this can be seen by applying the logarithm change of base). Applying these simplifications, we see that $\Theta(\lfloor \log_2(N) \rfloor) = \Theta(\log(N))$ just as we expected from our intuition.

Example Proof: Prove $\lfloor f(N) \rfloor = \Theta(f(N))$

Solution:

We start with the following inequality:

$$f(N) - \frac{1}{2} < \lfloor f(N) \rfloor \leq \lfloor f(N) \rfloor + \frac{1}{2} \leq f(N) + \frac{1}{2}$$

Simplifying $f(N) - \frac{1}{2}$ and $f(N) + \frac{1}{2}$ according to our big theta rules by dropping the constants, we see that they are of order $f(N)$. Therefore $\lfloor f(N) \rfloor + \frac{1}{2}$ is bounded by two expressions of order $f(N)$, and is therefore also $\Theta(f(N))$.

Exercise:

- Prove $\lceil f(N) \rceil = \Theta(f(N))$
- Prove $\log_p(N) = \Theta(\log_q(N))$

One cool fact to wrap up with: Log time is super good! It’s almost as fast as constant time, and way better than linear time. This is why we like binary search, rather than stepping one by one through our list and looking for the right thing.

To show this concretely:

N	$\log_2(N)$	Typical runtime (nanoseconds)
100	6.6	1
100,000	16.6	2.5
100,000,000	26.5	4
100,000,000,000	36.5	5.5
100,000,000,000,000	46.5	7

The input size increase by a factor of 1 trillion but the runtime only increased by a factor of about 7! This shows how remarkable $\log(N)$ runtime is especially for large size inputs.

13.3 Mergesort

In our last example, we’ll analyze merge sort, another cool sorting algorithm.

First, let’s remind ourselves of selection sort, which we will initially use as a building block for merge sort.

Selection sort works off two basic steps:

- Find the smallest item among the unsorted items, move it to the front, and ‘fix’ it in place.
- Sort the remaining unsorted/unfixed items using selection sort.

If we analyze selection sort, we see that its runtime is $\Theta(N^2)$.

Exercise: To convince yourself that selection sort has $\Theta(N^2)$ runtime, work through the geometric approach (try drawing out the state of the list at every sort call), or count the operations.

Let's introduce one other idea here: **arbitrary units of time**. While the exact time something will take will depend on the machine, on the particular operations, etc., we can get a general sense of time through our arbitrary units (AU).

If we run an $N = 6$ selection sort, and the runtime is of order N^2 , it will take ~ 36 AU to run. If $N = 64$, it'll take ~ 2048 AU to run. Now we don't know if that's 2048 nanoseconds, or seconds, or years, but we can get a relative sense of the time needed for each size of N .

Hold onto this thought for later analysis.

Now that we have selection sort, let's talk about **merging**.

Say we have two **sorted** arrays that we want to combine into a single big sorted array. We could append one to the other, and then re-sort it, but that doesn't make use of the fact that each individual array is already sorted. How can we use this to our advantage?

It turns out, we can merge them more quickly using the sorted property. The smallest element must be at the start of one of the two lists. So let's compare those, and put the smallest element at the start of our new list.

Now, the next smallest element has to be at the new start of one of the two lists. We can continue comparing the first two elements and moving the smallest into place until one list is empty, then copy the rest of the other list over into the end of the new list.

To see an animation of this idea, [go here](#).

What is the runtime of the merge operation? We can use the number of "write" operations to the new list as our cost model, and count the operations. Since we have to write each element of each list only once, the runtime is $\Theta(N)$.

Selection sort is slow, and merging is fast. How do we combine these to make sorting faster?

{% embed url="https://youtu.be/AlwAZkqzHqI" %} A closer look at Mergesort {% endembed %}

We noticed earlier that doing selection sort on an $N = 64$ list will take ~ 2048 AU. But if we sort a list half that big, $N = 32$, it only takes ~ 512 AU. That's more than twice as fast! So making the arrays we sort smaller has big time savings.

Having two sorted arrays is a good step, but we need to put them together. Luckily, we have merge. Merge, being of linear runtime, only takes ~ 64 AU. So in total, splitting it in half, sorting, then merging, only takes $512 + 512 + 64 = 1088$ AU. Faster than selection sorting the whole array. But how much faster?

Now, AUs aren't real units, but they're sometimes easier and more intuitive than looking at the runtime. The runtime for our split-in-half-then-merge-them sort is $\Theta(N \log N)$, which is about half of N^2 for selection sort. However, they are still both $\Theta(N^2)$.

What if we halved the arrays again? Will it get better? Yes! If we do two layers of merges, starting with lists of size $\frac{N}{4}$, the total time will be ~ 640 AU.

Exercise: Show why the time is ~ 640 AU by calculating the time to sort each sub-list and then merge them into one array.

What if we halved it again? And again? And again?

Eventually we'll reach lists of size 1. At that point, we don't even have to use selection sort, because a list with one element is already sorted.

This is the essence of **merge sort**:

- If the list is size 1, return. Otherwise:
- Mergesort the left half
- Mergesort the right half
- Merge the results

So what's the running time of **merge sort**?

We know merge itself is order N , so we can start by looking at each layer of merging:

- To get the top layer: merge ~ 64 elements = 64 AU
- Second layer: merge ~ 32 elements, twice = 64 AU
- Third layer: $\sim 16 \cdot 4 = 64$ AU
- ...

Overall runtime in AU is $\sim 64 \cdot k$, where k is the number of layers. Here, $k = \log_2(64) = 6$, so the overall cost of mergesort is ~ 384 AU.

Now, we saw earlier that splitting up more layers was faster, but still order N^2 . Is merge sort faster than N^2 ?

Yes!

Mergesort has worst case runtime $\Theta(N \log N)$.

- The top level takes $\sim N$ AU.
- Next level takes $\sim N/2 + \sim N/2 = \sim N$.

- One more level down: $\sim N/4 + \sim N/4 + \sim N/4 + \sim N/4 = \sim N$.

Thus, total runtime is $\sim Nk$, where k is the number of levels.

How many levels are there? We split the array until it is length 1, so $k = \log_2(N)$. Thus the overall runtime is $\Theta(N * \log(N))$.

Exercise: Use exact counts to argue for $\Theta(N * \log(N))$. Account for cases where we cannot divide the list perfectly in half.

So is $\Theta(N * \log(N))$ actually better than $\Theta(N^2)$? Yes! It turns out $\Theta(N * \log(N))$ is not much slower than linear time.

	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	4 sec
$n = 30$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	18 min	10^{25} years
$n = 50$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	11 min	36 years	very long
$n = 100$	< 1 sec	< 1 sec	< 1 sec	1 sec	12,892 years	10^{17} years	very long
$n = 1,000$	< 1 sec	< 1 sec	1 sec	18 min	very long	very long	very long
$n = 10,000$	< 1 sec	< 1 sec	2 min	12 days	very long	very long	very long
$n = 100,000$	< 1 sec	2 sec	3 hours	32 years	very long	very long	very long
$n = 1,000,000$	1 sec	20 sec	12 days	31,710 years	very long	very long	very long

timing_table_for_runtimes

13.4 Big O vs. Worst Case

Consider the following statements about BSTs. Which of the following are true?

1. The worst-case height of a BST is $\Theta(N)$.
2. BST height is $O(N)$.
3. BST height is $O(N^2)$.

The answer is that all three statements are true. BSTs always have a height that is linear or better, and a linear height is obviously “less than” the quadratic upper bound in the last point.

However, a more tricky question is which of the three statements is *the most informative*.

The answer here is the first statement: it gives an *exact* upper and lower bound unlike the other statements. $O(N)$ could mean linear, logarithmic, square-root, or constant, but $\Theta(N)$ can only mean linear.

For an analogy, consider the following statements about the worst-case cost of a hotel room:

1. The most expensive room is \$639/night.
2. The most expensive room is less than or equal to \$2000/night.

Here, we see that the first statement gives us exact information, whereas the second statement does not. In the second statement, the most expensive room could be \$2000, \$10, or anywhere in between.

However, *both are statements about the worst case*. Applying this to asymptotic notation, this means that we can refer to the worst case with Θ , O , or even Ω . **Big O is not the same as the worst case!**

Using Big O

If Θ is always more informative than O , then why do we bother using Big O notation at all? There are several reasons:

- We can make broader statements. For example, saying “binary search is $O(\log N)$ ” is correct, but saying “binary search tree is $\Theta(\log N)$ ” would not be correct, since it can be constant in certain scenarios.
- Sometimes, it is not possible or extremely difficult to determine the exact runtime. In such cases, we would still like to provide a generalized upper bound.

13. Asymptotics III